Postman and Nodemon :

- These are tools which help us to manage and test our backend Server and api .

> [!IMPORTANT]
> if there is any change after the server started then server can't replicate it , without refreshing it , so for this we use **nodemon** package which restart/refresh the server everytime any change happen

> [!IMPORTANT]
> Post man is a very powerfull Client tool which help us to Test our API , Server just works like a client and we don't need actual frontend for Backend development .

How nodemon Internally Works :

> [!CAUTION] https://www.perplexity.ai/search/what-is-middleware-in-node-js-nPklHrrfT8uHRbdK.hHppw?0=d&2=d#2
> **This Link is a thread of Proplexity , where i got a beautiful response about Internals of Nodemon .**

What is API (specific REST):

- **An API (Application Programming Interface) is a set of rules and protocols which helps us to interact with Differnt software or systems An API defines how requests and responses should be structured between a client (the requester) and a server (the responder), establishing a contract that both sides follow to ensure proper communication.**
- **It hides the internal workings of a system, exposing only the necessary parts for interaction, which simplifies programming and enhances security by limiting access to only relevant data or functions.**
- **APIs are used extensively in modern technology to connect mobile apps, cloud services, websites, payment gateways, social media platforms, and more, enabling developers to build complex applications faster by leveraging existing services.**

REST API :

Absolutely, Soumya! Let me explain **REST API** in the **simplest possible way**—like you're telling a friend who's new to web development.

🧠 What is an API?

API (Application Programming Interface) is like a **waiter** in a restaurant:

- You (the client) ask for something (e.g., menu item),
- The waiter (API) takes your request to the kitchen (server),
- Brings back the food (response).

🌐 What is a REST API then?

REST stands for **Representational State Transfer**. It's a **style** or **rules** for designing APIs that are:

- Simple
- Fast
- Work over the **web (HTTP)**

🎯 Key Features (Uniqueness) of REST API:

Feature	Explained Simply
Stateless	Each request is **independent**. The server doesn't remember what you did before. Like ordering food again—you tell the waiter everything from scratch.
Uses HTTP Methods	REST uses common web methods like: `GET` (read), `POST` (create), `PUT` (update), `DELETE` (remove).
Uses URLs	Everything is accessed through a **URL**. For example:

Feature	Explained Simply
	`/users` → list users `/users/5` → get user with ID 5
Data is usually JSON	REST APIs **send and receive data in JSON**, which is lightweight and easy to read.
Client-Server Separation	Frontend (React, app) and backend (Node.js, DB) are separate. They talk through REST APIs.

🧊 Real-Life Example:

Let's say you're building a **college app** and you want to get student data.

Your frontend (React) will call:

```
GET https://collegeapi.com/students
```

The server responds with:

```
[
  { "id": 1, "name": "Soumya" },
  { "id": 2, "name": "Ankit" }
]
```

That's a **REST API** at work!

💡 Why REST is Popular?

✅ Easy to understand
✅ Works with any language (Python, Node, Java, etc.)
✅ Scalable and flexible
✅ Web-friendly (uses HTTP and URLs)

💯 WHAT IS REST API (asked by interviewr):

Great question, Soumya! Here's how you can **confidently answer** the interview question:
"What is a REST API and what makes it unique?" — in a **clear, concise, and professional** way 👇

✅ Sample Interview Answer (Simple + Smart)

> **"A REST API, or Representational State Transfer API, is a set of rules that allows different software applications to communicate with each other over the web using standard HTTP methods like GET, POST, PUT, and DELETE.**
>
> **What makes REST APIs unique is that they are stateless, meaning each request from the client contains all the information needed, and the server doesn't store any client session data. REST APIs are also resource-based — data is represented as resources accessible via unique URLs.**
>
> **REST is lightweight, flexible, and widely used in web development, especially for frontend-backend communication, because it's simple and works well over HTTP using JSON for data transfer."**

✍️ Bonus Tip (If They Ask for Example):

> **"For example, in a college app, if we want to fetch student data, we might use a GET request like `/students/1` to get the details of a student with ID 1. This kind of URL-driven and method-based interaction is what makes REST APIs easy to design and consume."**

How to send Data from Frontend to Backend `

- There are 3 way we majorly send information from **Frontend(client)** to the **Backend(server / API)**

> **Example:- req.body(), req.param(), req.query()**

These three — `req.body`, `req.params`, and `req.query` — are **used in Express.js** (Node.js framework) to **get data sent from the frontend to the backend**.

Let's break them down **super simply**, with **real-life examples** 👇

🔷 1. `req.body`

✅ Use: When sending data in the body (usually in a POST or PUT request)

📦 Example:

Frontend (React or Postman):

```
POST /register
Content-Type: application/json

{
  "name": "Soumya",
  "email": "soumya@gmail.com"
}
```

Backend (Express.js):

```
app.post('/register', (req, res) => {
  const name = req.body.name;
  const email = req.body.email;
  res.send(`Welcome ${name}, your email is ${email}`);
});
```

🔧 **Make sure to use**:

```
app.use(express.json());
```

🔷 2. `req.params`

✅ Use: When sending data in the URL itself, like `/user/:id`

📦 Example:

Frontend hits:

```
GET /user/101
```

Backend:

```
app.get('/user/:id', (req, res) => {
  const userId = req.params.id;
  res.send(`User ID is ${userId}`);
});
```

Here `:id` is a **route parameter** — passed as part of the URL path.

🔷 3. `req.query`

✅ Use: When sending optional data using ?key=value in the URL

📦 Example:

Frontend hits:

```
GET /search?keyword=javascript&limit=10
```

Backend:

```
app.get('/search', (req, res) => {
  const keyword = req.query.keyword;
  const limit = req.query.limit;
  res.send(`You searched for ${keyword} with limit ${limit}`);
});
```

📊 Summary Table:

Feature	Data Sent Through	Method Type	Example
`req.body`	Request body (JSON)	`POST`, `PUT`	`{ "name": "Soumya" }`
`req.params`	URL path (dynamic part)	`GET`, `DELETE`	`/user/101`
`req.query`	URL query string	`GET`	`/search?keyword=js&limit=10`